



# Database Structures & Data Logic

Week 5 • Day 1 • Technical Infrastructure & Modeling

TPM Academy



# Day 1 Objectives

- Read a data model at the entity level: **keys, indexes, relationships, normalization vs denormalization**
- Apply the "**access pattern first**" discipline — queries before schema
- Make the **storage choice** argument in customer terms
- Surface **schema-level trade-offs** using the Week-4 template
- Draft **TMD §1 — Data model**



# Today's Shape

| Clock         | Block                       | Outcome                      |
|---------------|-----------------------------|------------------------------|
| 09:00 – 09:15 | Opening                     | Pin TCD §§1–6 on the wall    |
| 09:15 – 10:30 | Block 1 + Activity 1        | Read & write access patterns |
| 10:45 – 12:00 | Block 2 + Activity 2        | Draft the entity model       |
| 13:00 – 14:15 | Block 3 + Activity 3        | Storage trade-offs           |
| 14:30 – 16:00 | Block 4 + Activity 4 + Wrap | AI critique + polish         |

# Block 1 — Access Patterns First

Queries before schema



# The Vocabulary Card

| Term            | What it means                                                    |
|-----------------|------------------------------------------------------------------|
| Entity          | A "thing" the system tracks (Dispatcher, Ticket, ReconcileEvent) |
| Primary key     | The unique identifier for an entity                              |
| Foreign key     | A reference from one entity to another                           |
| Index           | Speed up reads at the cost of writes + storage                   |
| Normalization   | Splitting data to avoid duplication                              |
| Denormalization | Duplicating data on purpose for read speed                       |
| Cardinality     | 1:1, 1:N, N:N — the relationship shape                           |
| Constraint      | A database-enforced rule (NOT NULL, UNIQUE, CHECK)               |



# The Four Storage Buckets

| Bucket                                    | When to reach for it                                                  |
|-------------------------------------------|-----------------------------------------------------------------------|
| <b>Relational</b> (Postgres, MySQL)       | Strong relationships, transactions, complex queries — default for B2B |
| <b>Document</b> (MongoDB, Firestore)      | Flexible schema, hierarchical data; shape varies per record           |
| <b>Key-value</b> (Redis, DynamoDB)        | High throughput, simple lookups, caches, sessions                     |
| <b>Search</b> (Elasticsearch, OpenSearch) | Full-text search, analytics on text                                   |

**The TPM doesn't pick the engine.** The TPM describes the access pattern clearly enough that the architect can.



# The "Access Pattern First" Discipline

1. **List the queries the feature needs.** "Show me a dispatcher's open tickets, sorted by latest activity."
2. **For each query: what does it read, in what order, with what filter?**
3. **Now design the schema** so the queries are cheap.

**If your data model can't be defended by the queries that drive it, it's wrong.**

# Activity 1 — Read & Write Patterns

Triads • 35 minutes

List every read and write the feature needs. Capture data, frequency, freshness requirement.

- 5 min: re-read PRD §5
- 15 min: list reads (data, filter, freshness)
- 10 min: list writes (entity, invariants, concurrency)
- 5 min: highlight 1–2 highest-volume reads



5 + 15 + 10 + 5



# Block 2 — Draft the Entity Model

Tables, keys, indexes, relationships



# The Entity Template

```
## Entity: <name>

<table>
<thead>
<tr>
<th>Field</th>
<th>Type</th>
<th>Notes</th>
</tr>
</thead>
<tbody><tr>
<td>id</td>
<td>UUID</td>
<td>Primary key</td>
</tr>
<tr>
<td>...</td>
<td>...</td>
<td>NOT NULL? UNIQUE? FK to ...?</td>
</tr>
</tbody></table>
<p><strong>Indexes:</strong></p>
<ul>
<li>(field, field) – purpose: query #N from access patterns</li>
</ul>
<p><strong>Relationships:</strong></p>
<ul>
<li>belongs_to <entity> via <fk_field></fk_field></entity></li>
<li>has_many <entity> via <fk_field></fk_field></entity></li>
</ul>
<p><strong>Notes / invariants:</strong></p>
</name>
```

```
<li>A reconcile_event is immutable once created.</li>
```

- ticket.status ∈ {open, reconciled, voided}.

**Indexes reference access patterns by number.** "Index for query #4" beats "useful index."



# Worked Entity (FieldPulse)

```
## Entity: ReconcileEvent
```

```
<table>
<thead>
<tr>
<th>Field</th>
<th>Type</th>
<th>Notes</th>
</tr>
</thead>
<tbody><tr>
<td>id</td>
<td>UUID</td>
<td>PK</td>
</tr>
<tr>
<td>dispatcher_id</td>
<td>UUID</td>
<td>FK → Dispatcher</td>
</tr>
<tr>
<td>ticket_ids</td>
<td>UUID[]</td>
<td>The set reconciled (denormalized)</td>
</tr>
<tr>
<td>submitted_at</td>
<td>timestamp</td>
<td>NOT NULL</td>
</tr>
<tr>
<td>client_ip</td>
<td>inet</td>
<td>for audit</td>
```

```
<li>Immutable: never UPDATE'd after INSERT</li>
```

- ticket\_ids is denormalized (see trade-off #2)

## Activity 2 — Entity Model

Triads • 40 minutes

List entities. For each, draft fields, indexes, relationships, invariants.

- 10 min: list entities (new + existing)
- 15 min: draft fields per new entity
- 10 min: indexes (referencing patterns)
- 5 min: relationships + cardinalities



10 + 15 + 10 + 5

# Block 3 — Storage Trade-Offs

Use the Week-4 trade-off template at schema scale

# Three Schema-Level Trade-Offs

| Trade-off                              | Tension                                        |
|----------------------------------------|------------------------------------------------|
| Normalization vs denormalization       | Clean writes vs fast reads                     |
| Strong consistency vs replica reads    | Always-fresh vs scalable read load             |
| Schema flexibility vs schema integrity | Easy migration vs caught-by-database integrity |

**Same Week-4 template:** Option A vs Option B / Choice / Why / Accepted cost / Revisit trigger.



# Worked Schema Trade-Off

```
### Trade-off – ticket_ids storage
**Tension:** Normalization vs denormalization.
**Option A:** ReconcileEvent.ticket_ids: UUID[] (denormalized).
**Option B:** Separate ReconcileEventTicket join table.
**Choice:** Option A.
**Why:** Reconcile events are read in batches of 1 most of the
time. Reading the array is one row. Option B requires
a join. Reconcile events are immutable and rarely
queried by ticket – access patterns don't reward
normalization.
**Accepted cost:** Cannot index by individual ticket_id (no
"find every reconcile that touched ticket X" query
without a denormalization).
**Revisit trigger:** If "find reconciles touching ticket X"
becomes a high-volume query.
```



# Activity 3 — Storage Trade-Offs

Triads • 40 minutes

- 15 min: identify 3 trade-off points in your model
- 15 min: write each (Option A/B/Choice/Why/Cost/Trigger)
- 10 min: cross-check against TCD §4 SLOs



15 + 15 + 10

# Block 4 — AI Critique + Polish

Treat the AI as a critic, not an oracle



# Today's Two Prompts

A. Critique this schema

Role: Senior backend engineer reviewing a feature's data model.

Context: <entity definitions="" +="" indexes="" relationships="" access="" pattern="" sheet="">

Task: Identify the top 3 issues.

Constraints:

- Name the specific access pattern or invariant that breaks
- Suggest a concrete fix (new index / entity / cardinality)
- Do not suggest generic best practices

Format: Numbered – Issue / What breaks / Fix.

</entity>

B. What did we forget?

Continuing from above. Given these access patterns, what queries the user will eventually want are NOT yet supported by this schema? List 3, ranked by likelihood within 6 months.

# Activity 4 — AI Critique

Triads • 45 minutes • Wrap

- 10 min: Prompt A → 3 issues
- 10 min: adopt / defer / reject
- 10 min: Prompt B → forgotten queries
- 15 min: update §1 + provenance + AI-prose check



10 + 10 + 10 + 15 · 15-min wrap



# Day 1 Wrap

## What we built

- Access Pattern Sheet
- Entity model (PKs, indexes, relationships)
- 3+ explicit storage trade-offs
- TMD §1 drafted

## What opens tomorrow

- **Cloud architecture & infrastructure**
- Region / AZ / managed services
- Multi-tenancy stance

**Bring to Day 2:** Your TMD §1 + the TCD's Container diagram. Tomorrow we put the boxes in regions.

# Questions & Reflection

Which of your indexes would you cut if writes became 10x more expensive?